

ACTIVITY 5: Introducing Exceptions in the Account Class

Objective

Enhance the `Account` class by replacing boolean returns with proper exception handling.

Why Exceptions?

- **Professional:** Real-world applications use exceptions
- **Informative:** Exceptions carry detailed error messages
- **Enforceable:** Cannot ignore exceptions like boolean returns
- **Cleaner:** Separates error handling from business logic

Requirements

Key Changes from Boolean Version

1. **Remove all boolean returns**
2. **Throw exceptions** for all error conditions
3. **Custom exceptions** for different error types
4. **Constructor throws exceptions** instead of self-correcting

Custom Exception Classes

Create these custom exception classes:

1. **AccountException** - Base exception for all account-related errors
2. **InsufficientBalanceException** - When balance is insufficient
3. **InvalidAmountException** - When amount is invalid (negative or zero)
4. **InactiveAccountException** - When operation attempted on inactive account
5. **InvalidPinException** - When PIN is incorrect or not set
6. **MinimumBalanceViolationException** - When withdrawal would violate minimum balance

Account Class Methods (With Exceptions)

Method	Parameters	Return Type	Exceptions Thrown
deposit	amount (double)	void	InvalidAmountException, InactiveAccountException
withdraw	amount (double), pin (int)	void	InvalidAmountException, InsufficientBalanceException, MinimumBalanceViolationException, InactiveAccountException, InvalidPinException
closeAccount	None	void	IllegalStateException (if already closed)
reopenAccount	None	void	IllegalStateException (if already active)
setPin	pin (int)	void	IllegalArgumentException (if invalid PIN)

Constructor

```
public Account(int accountNumber, String name, int age,  
              double initialBalance, String accountType)
```

Throws `IllegalArgumentException` for:

- Age < 18
- Invalid account type
- Balance below minimum

Important Notes

- **No boolean returns** - all methods return void
- **No self-correction** - constructor throws exceptions
- **Specific exceptions** - different types for different errors

Task 1: Create Custom Exception Classes

Create these exception classes in separate files:

AccountException.java

```
/**
 * Base exception for all account-related errors.
 */
public AccountException extends Exception {

    public AccountException(String message) {
        super(message);
    }
}
```

InvalidAmountException.java

```
/**
 * Thrown when an invalid amount is provided (negative or zero).
 */
public InvalidAmountException extends AccountException {

    public InvalidAmountException(String message) {
        super(message);
    }
}
```

InsufficientBalanceException.java

```
/**
 * Thrown when attempting to withdraw more than available balance.
 */
public InsufficientBalanceException extends AccountException {

    public InsufficientBalanceException(String message) {
        super(message);
    }
}
```

MinimumBalanceViolationException.java

```
/**
 * Thrown when withdrawal would violate minimum balance requirement.
 */
public MinimumBalanceViolationException extends AccountException {

    public MinimumBalanceViolationException(String message) {
        super(message);
    }
}
```

InactiveAccountException.java

```
/**
 * Thrown when operation is attempted on an inactive account.
 */
public InactiveAccountException extends AccountException {

    public InactiveAccountException(String message) {
        super(message);
    }
}
```

InvalidPinException.java

```
/**  
 * Thrown when PIN is incorrect or not set.  
 */  
public InvalidPinException extends AccountException {  
  
    public InvalidPinException(String message) {  
        super(message);  
    }  
}
```

Task 2: Create the Enhanced Account Class with Exceptions

Write the complete `Account` class that uses exceptions instead of boolean returns:

```
public Account {

    // ===== Constants =====
    private static final double MIN_BALANCE_SAVINGS = 500.0;
    private static final double MIN_BALANCE_CURRENT = 1000.0;
    private static final int MIN_AGE = 18;
    private static final int MIN_PIN = 1000;
    private static final int MAX_PIN = 9999;

    // ===== Fields =====
    private int accountNumber;
    private String name;
    private int age;
    private double balance;
    private String accountType;
    private String status;
    private Integer pin;

    // ===== Constructor =====
    public Account(int accountNumber, String name, int age,
                   double initialBalance, String accountType)
        throws IllegalArgumentException {
        // TODO: Validate age (must be >= 18)
        // TODO: Validate account type (must be "Savings" or "Current")
        // TODO: Validate minimum balance based on account type
        // TODO: Initialize all fields
        // TODO: Set status to "Active"
        // TODO: Set pin to null
    }

    // ===== Business Methods =====
    public void deposit(double amount)
        throws InvalidAmountException, InactiveAccountException {
        // TODO: Check if account is active
        // TODO: Check if amount is positive
    }
}
```

```
// TODO: Add amount to balance
}

public void withdraw(double amount, int pin)
    throws InvalidAmountException,
           InsufficientBalanceException,
           MinimumBalanceViolationException,
           InactiveAccountException,
           InvalidPinException {
    // TODO: Check if account is active
    // TODO: Check if PIN is set
    // TODO: Verify PIN
    // TODO: Check if amount is positive
    // TODO: Check if sufficient balance
    // TODO: Check minimum balance after withdrawal
    // TODO: Deduct amount from balance
}

// ===== Account Status Management =====
public void closeAccount() throws IllegalStateException {
    // TODO: Check if already closed
    // TODO: Set status to "Inactive"
}

public void reopenAccount() throws IllegalStateException {
    // TODO: Check if already active
    // TODO: Set status to "Active"
}

// ===== PIN Management =====
public void setPin(int pin) throws IllegalArgumentException {
    // TODO: Validate PIN (4-digit number)
    // TODO: Set pin
}

public boolean verifyPin(int pin) {
    // TODO: Return true if PIN matches, false otherwise
}
```

```
public boolean hasPin() {  
    // TODO: Return true if PIN is set  
}  
  
// ===== Helper Methods =====  
private double getMinimumBalance() {  
    // TODO: Return minimum balance based on account type  
}  
  
private void validateActive() throws InactiveAccountException {  
    // TODO: Throw InactiveAccountException if not active  
}  
  
// ===== Getters =====  
// TODO: Add all getters  
}
```